System przetwarzania danych oparty na pojedynczym oraz współpracujących modelach LLM

Oliwier Komorowski Jakub Nowak

5 listopada 2026

Streszczenie

Projekt inżynierski dotyczy współpracy wielu dużych modeli językowych (LLM) w rozwiązywaniu złożonych problemów.

Słowa kluczowe: duży model językowy (LLM), współpraca wielu modeli, umysł rojowy, Ollama
Dziedzina nauki i techniki, zgodnie z wymogami OECD: Nauki o komputerach i informatyka

Abstract

This engineering project concerns the cooperation of multiple large language models (LLMs) in solving complex problems.

Keywords: large language model (LLM), multi-model collaboration, hive mind, Ollama
Field of science and technology in accordance with OECD requirements: Computer Science and Information technology

Spis treści

1 Wstęp i cel pracy **5**
- 1.1 Motywacja . . . 5
  - 1.1.1 Dynamiczny rozwój dziedziny . . . 5
  - 1.1.2 Nowe możliwości . . . 6
- 1.2 Cel pracy . . . 7
- 1.3 Struktura pracy . . . 8

2 Wprowadzenie do dziedziny **9**
- 2.1 Generatywna sztuczna inteligencja . . . 9
  - 2.1.1 Definicja i miejsce generatywnej sztucznej inteligencji . . . 9
  - 2.1.2 Zasada działania i zastosowania . . . 9
  - 2.1.3 Ograniczenia generatywnej sztucznej inteligencji . . . 10
- 2.2 Duże modele językowe . . . 10
  - 2.2.1 Definicja i architektura . . . 10
  - 2.2.2 Uczenie dużych modeli językowych . . . 11
  - 2.2.3 Sterowanie, wnioskowanie i wyzwania wdrożeniowe . . . 12
- 2.3 Mechanizmy współpracy . . . 13
  - 2.3.1 Głosowanie . . . 13
  - 2.3.2 Weryfikacja odpowiedzi przez drugi model . . . 13
  - 2.3.3 Debata . . . 13

3 Technologie i narzędzia **15**
- 3.1 Wybrane języki programowania . . . 15
- 3.2 Biblioteki . . . 15
- 3.3 Wybrane modele językowe . . . 16
- 3.4 Narzędzia . . . 18
- 3.5 Wymagania sprzętowe . . . 19

4 Projekt systemu **21**
- 4.1 Implementacja systemu przetwarzania z wykorzystaniem pojedynczego modelu LLM . . . 21
- 4.2 Rozszerzenie systemu do wariantu z dwoma współpracującymi modelami LLM 21

5 Badania i analiza **22**
- 5.1 Wybór benchmarków oraz metryk oceny jakości odpowiedzi . . . 22
  - 5.1.1 Benchmarki . . . 22
  - 5.1.2 Metryki oceny . . . 22
- 5.2 Przeprowadzenie analizy wyników oraz porównanie jakości odpowiedzi w obu trybach działania systemu . . . 22

6 Podsumowanie **23**

Rozdział 1

Wstęp i cel pracy

Aktualnie duże model językowe (ang. LLM, Large Language Model) wykorzystywane są w różnych obszarach obszarach życia. Wysokie zainteresowanie sztuczną inteligencją sprzyja rozwojowi tej dziedziny. Skutkuje to pojawianiem się nowych możliwości wykorzystania technologii przetwarzania języka naturalnego oraz zrewolucjonizowaniem sposobu, w jakim systemy informatyczne przetwarzają i analizują dane nieustrukturyzowane.

W miarę jak rosną oczekiwania wobec systemów opartych na LLM, pojedynczy model może okazać się niewystarczający. Typowymi ograniczeniami są koszt obliczeniowy pojedynczego zapytania czy podatność na błędy wynikające z braku weryfikacji własnych odpowiedzi. Naturalną odpowiedzią na te ograniczenia jest koncepcja architektur wielomodelowych, w których kilka instancji LLM współpracuje ze sobą.

1.1 Motywacja

Motywacją do realizacji projektu jest rosnące zainteresowanie sztuczną inteligencją, w szczególności dużymi modelami językowymi i ich możliwościami. Jednak główną motywacją jest sprawdzenie, czy dwa różne modele współpracując udzielą lepszych odpowiedzi niż pojedyńczy model.

1.1.1 Dynamiczny rozwój dziedziny

W ciągu ostatnich dekad można było zauważyć dynamiczny postęp w obszarze sztucznej inteligencji (SI, ang. Artificial Intelligence, AI). Rozwój tej dziedziny doprowadził do znacznego zwiększenia liczby prowadzonych na całym świecie badań, co znajduje odzwierciedlenie między innymi w rosnącej liczbie cytowań publikacji poświęconych tej tematyce. W ciągu ostatniej dekady sztuczna inteligencja (SI) rozwija się bardzo dynamicznie. Przełom przyniósł koniec 2022 roku, kiedy publicznie udostępniono ChatGPT. Prostota obsługi także dla osób bez wiedzy technicznej przyczyniła się do jego błyskawicznej popularności [1]. Szybki rozwój badań nad dużymi modelami językowymi (LLM) potwierdza przegląd ponad 5000 publikacji z lat 2017-2023 [2]. Generatywna SI (GenAI) znajduje zastosowanie m.in. w medycynie, edukacji, biznesie, inżynierii i tworzeniu oprogramowania oraz w generowaniu treści, w tym muzyki i wideo [1], [2].

Jedną z dziedzin rozwijających się w ramach sztucznej inteligencji jest przetwarzanie języka naturalnego (ang. Natural Language Processing, NLP). Jej głównym zadaniem jest umożliwienie komputerom pracy z językiem używanym przez człowieka poprzez jego odpowiednie

przekształcanie i interpretowanie. Pozwala to na automatyczną analizę, generowanie oraz przetwarzanie treści tekstowych. Istotnym elementem rozwoju NLP stały się również duże modele językowe (LLM), których możliwości w ciągu ostatnich lat znacząco wzrosły. Modele te ewoluowały z pasywnych systemów przetwarzania tekstu w aktywne podmioty (agenty AI) zdolne do autonomicznego planowania, rozumowania i wchodzenia w interakcje ze środowiskiem [3]. Ich popularność gwałtownie zwiększyła się w listopadzie 2022 roku, kiedy firma OpenAI udostępniła model ChatGPT-3.5. Narzędzie to w bardzo krótkim czasie zdobyło ogromną liczbę użytkowników - w nieco ponad dwa miesiące przekroczyło próg stu milionów użytkowników.

Coraz częściej duże modele językowe służą również do wyszukiwania informacji i rozwiązywania problemów. Przykładowo generatywna sztuczna inteligencja (GenAI) może pomóc rozwiązać zadanie matematyczne, wytłuamczyć zagadnienie naukowe, czy udzielić odpowiedzi na pytanie o zdrowie. W zależności od zapytania i sposobu jego sformułowania model dopasowywuje sposób odpowiedzi. Zamiast samego wyniku potrafi przedstawić sobób w jaki do niego doszedł lub dostosować wypowiedź do wymaganego poziomu. Ze względu na to ważne jest sformułowanie odpowiedniego promptu. Zbyt ogólne polecenie lub brak kontekstu może skończyć się odpowiedzią niepełną, błędną albo inną niż oczekiwana. Z halucynacjami próbuje się walczyć za pomocą wielowarstwowych zabezpieczeń i monitorowania przebiegu pracy (Workflow Monitoring). Wymuszenie odpowiedzi w ustrukturyzowanym formacie (JSON) ułatwia parsowanie, kontrolę struktury i walidację zgodności ze schematem[4]. Halucynacje mają duże znaczenie tam, gdzie liczy się dokładność, choćby w matematyce czy medycynie. Odpowiedzi modelu należy tam traktować jako wsparcie, a nie w pełni wiarygodne źródło.

1.1.2 Nowe możliwości

Dynamiczny rozwój sztucznej inteligencji oraz metod przetwarzania języka naturalnego przyczynił się do powstania nowych sposobów wykorzystania komputerów w codziennych zadaniach. Szczególne znaczenie mają tutaj generatywna sztuczna inteligencja oraz duże modele językowe, które ewoluowały z pasywnych systemów odpowiedzi w autonomiczne agenty AI. Umożliwiają one nie tylko uzyskiwanie informacji, ale również samodzielne korzystanie z zewnętrznych narzędzi, planowanie wieloetapowych zadań i interakcję ze środowiskiem w celu realizacji złożonych celów [5]. Rozwiązania te mogą być wykorzystywane między innymi do wspomagania nauki, rozwiązywania zadań matematycznych, wyjaśniania zagadnień z różnych dziedzin czy udzielania odpowiedzi na pytania wymagające analizy dostarczonych informacji.

Duże modele językowe

Duże modele językowe (ang. Large Language Models, LLM) w ostatnich latach stały się jednym z najważniejszych obszarów rozwoju sztucznej inteligencji. Jeszcze stosunkowo niedawno bezpośrednia komunikacja z komputerem za pomocą języka naturalnego wymagała specjalistycznych rozwiązań, natomiast obecnie użytkownik może formułować pytania i polecenia w sposób zbliżony do rozmowy z drugą osobą. Dzięki temu modele językowe mogą pełnić funkcję narzędzia wspomagającego, a obecnie coraz częściej tworzą złożone systemy wieloagentowe (ang. Large Language Model-based Multi-Agent Systems, MASs) [3], [5]. W takich strukturach wyspecjalizowane jednostki (np. agent-programista, agent-tester) współpracują ze sobą, co pozwala na naturalny podział pracy, większą odporność na błędy oraz rozwiązywanie problemów o stopniu złożoności przekraczającym możliwości pojedynczego modelu. Przykładowe zastosowania obejmują generowanie i analizowanie tekstu, tłumaczenie treści, tworzenie podsumowań, odpowiadanie na pytania, a także pomoc w rozwiązywaniu problemów matematycznych i logicznych.

Istotną zaletą pewnej części dostępnych modeli językowych jest ich otwarty charakter (ang. open-source software), który umożliwia wykorzystanie ich we własnych aplikacjach. Platformy takie jak Hugging Face udostępniają szeroki wybór modeli, które można dostosować do określonych zastosowań i zintegrować z tworzonym oprogramowaniem. Pozwala to między innymi na budowanie systemów wykorzystujących modele językowe do analizy pytań użytkownika oraz generowania odpowiedzi bez konieczności tworzenia własnego modelu od podstaw.

Wykorzystanie modeli uruchamianych lokalnie daje również możliwość ograniczenia konieczności przesyłania danych do zewnętrznych usług. Przy odpowiednio wydajnej infrastrukturze model może zostać uruchomiony bezpośrednio na komputerze użytkownika, a komunikacja z nim może odbywać się za pośrednictwem lokalnego interfejsu API. Takie rozwiązanie może mieć szczególne znaczenie w przypadku aplikacji przetwarzających informacje, dla których istotne są prywatność i bezpieczeństwo. Lokalna obsługa modelu pozwala bowiem zachować dane w obrębie własnej infrastruktury, ograniczając ryzyko ich nieautoryzowanego udostępnienia.

1.2 Cel pracy

Celem pracy jest implementacja systemu przetwarzania danych opartego na modelach LLM, działającego w dwóch wariantach: z wykorzystaniem pojedynczego modelu LLM oraz z wykorzystaniem dwóch modeli LLM. Projekt zakłada implementację takiego systemu. Głównym zamysłem pracy jest zbadanie wpływu współpracy wielu modeli na jakość odpowiedzi oraz identyfikację najlepszych praktyk w zakresie projektowania i implementacji takich rozwiązań.

Aplikacja zostanie zaimplementowana w języku Python i będzie umożliwiała komunikację z lokalnie uruchomionymi dużymi modelami językowymi. Połączenie z modelami zostanie zrealizowane za pośrednictwem interfejsu API platformy Ollama, dzięki czemu przetwarzanie zapytań będzie odbywało się w całości na urządzeniu użytkownika, bez konieczności korzystania z zewnętrznych usług chmurowych. System będzie działał w dwóch wariantach. W wariancie bazowym zadanie rozwiąże jeden model, bez udziału innych. Posłuży on jako punkt odniesienia przy ocenie, czy współpraca dwóch modeli daje lepsze wyniki niż praca jednego.

W wariancie z dwoma modelami współpraca zostanie ograniczona do prostego schematu debaty. Najpierw oba modele rozwiążą zadanie niezależnie. Ich odpowiedzi zostaną następnie znormalizowane, czyli sprowadzone do wspólnej postaci, tak aby można je było porównać. Jeśli będą zgodne, system zwróci tę odpowiedź bez debaty. W przeciwnym razie rozpocznie się debata, w której modele będą wymieniać argumenty i kontrargumenty najwyżej przez ustaloną z góry liczbę rund. Zgodność odpowiedzi będzie sprawdzana po każdej rundzie, a debata zakończy się, gdy tylko modele dojdą do porozumienia. Jeśli nie dojdą do niego do ostatniej rundy, system zwróci informację o braku zgodnej odpowiedzi.

Wspólną przestrzenią modeli będzie zestaw ich aktualnych rozwiązań. W każdej rundzie debaty model otrzyma treść zadania oraz rozwiązania obu modeli z poprzedniej rundy, czyli własne i rozwiązanie drugiego modelu, każde z odpowiedzią końcową i uzasadnieniem. Na tej podstawie podtrzyma swoją odpowiedź albo ją zmieni. Kontekst będzie budowany od nowa w każdej rundzie, więc model zobaczy tylko ostatnie stanowiska, a nie całą historię debaty. Oba modele będą odpowiadały w danej rundzie na podstawie tych samych danych, niezależnie od kolejności wywołań.

Oba warianty otrzymają te same zadania i parametry generowania, co pozwoli porównać je bezpośrednio.

Interakcja z aplikacją będzie odbywała się z poziomu terminala za pomocą tekstowego interfejsu wiersza poleceń (ang. Command Line Interface, CLI). Aplikacja będzie umożliwiać

podgląd aktualnego stanu przetwarzania za pomocą logów generowanych w trakcie działania systemu i wyświetlania ich użytkownikowi na bieżąco. Odpowiedź końcowa również zostanie zaprezentowana w terminalu. Główny nacisk zostanie położony na funkcjonalność i logikę działania usługi, a nie na rozbudowaną warstwę prezentacji.

Projekt ma na celu zbadanie efektywności współpracy wielu dużych modeli językowych w rozwiązywaniu złożonych problemów oraz identyfikację najlepszych praktyk w zakresie projektowania i implementacji systemów opartych na współpracujących LLM.

1.3 Struktura pracy

Praca liczy sześć głównych rozdziałów, ułożonych tak, aby krok po kroku poznawać zagadnienie oraz kolejne etapy realizacji projektu. W rozdziale pierwszym przedstawiono tematykę pracy i określono jej cel. Rozdział drugi ma charakter teoretyczny i wprowadza w omawianą dziedzinę, wyjaśniając pojęcie generatywnej sztucznej inteligencji oraz dużych modeli językowych. W rozdziale trzecim opisano technologie użyte w projekcie, do których należą Python, Ollama i modele językowe. Rozdział czwarty poświęcono projektowi systemu, w którym szczegółowo omówiono poszczególne komponenty. W rozdziale piątym zaprezentowano wyniki badań dotyczące skuteczności współpracy wielu modeli, wydajności modeli językowych oraz jakości generowanych przez nie odpowiedzi. Rozdział ostatni zawiera podsumowanie uzyskanych rezultatów oraz kierunki dalszego rozwoju pracy.

W tabeli 1.1 przedstawiono podział odpowiedzialności autorów za poszczególne rozdziały i podrozdziały.

Tabela 1.1: Podział odpowiedzialności autorów za poszczególne rozdziały

Autor	**Numery rozdziałów i podrozdziałów**
Olwier Komorowski	1, 2.2, 3.1, 3.2, 3.4, 4.1, 5.1
Jakub Nowak	2.1, 3.3, 3.5, 4.2, 5.2, 6

Rozdział 2

Wprowadzenie do dziedziny

Niniejszy rozdział wprowadza w dziedzinę, w której osadzona jest praca. Najpierw przedstawia generatywną sztuczną inteligencję: jej definicję, zasadę działania, zastosowania oraz ograniczenia. Następnie omawia duże modele językowe, czyli klasę modeli generatywnych, na której opiera się system MLLMS. Wprowadzone pojęcia są podstawą dla kolejnych rozdziałów, które opisują technologie, projekt systemu i wyniki badań.

2.1 Generatywna sztuczna inteligencja

2.1.1 Definicja i miejsce generatywnej sztucznej inteligencji

Sztuczna inteligencja (SI) obejmuje metody, które pozwalają komputerom wykonywać zadania wymagające zwykle ludzkiej inteligencji. Jej istotną częścią jest uczenie maszynowe (ang. Machine Learning, ML), w którym zależności nie są programowane wprost, lecz wyznaczane przez algorytm na podstawie danych. Szczególnym przypadkiem uczenia maszynowego są sztuczne sieci neuronowe (ang. Artificial Neural Networks), czyli modele zbudowane z warstw prostych jednostek obliczeniowych, których parametry dobierane są w procesie uczenia. Sieci składające się z wielu warstw określa się mianem uczenia głębokiego (ang. Deep Learning, DL). Umożliwiło ono modelowanie bardzo złożonych zależności w danych, takich jak tekst, obraz czy dźwięk.

Na uczeniu głębokim opiera się współczesna generatywna sztuczna inteligencja (ang. Generative Artificial Intelligence, GenAI). Według [6] są to techniki obliczeniowe zdolne do generowania pozornie nowych, sensownych treści, takich jak tekst, obrazy czy dźwięk, na podstawie danych treningowych. Modele generatywne nie ograniczają się więc do klasyfikowania danych wejściowych lub przewidywania pojedynczej wartości, lecz tworzą nowe dane, przypominające te, na których były uczone. Wśród nich szczególne miejsce zajmują duże modele językowe, opisane w sekcji 2.2.3.

2.1.2 Zasada działania i zastosowania

Modele generatywne uczą się rozkładu prawdopodobieństwa danych treningowych $p(x)$, a w wersji warunkowej rozkładu $p(x \mid c)$, w którym c oznacza kontekst, na przykład polecenie użytkownika (ang. prompt). Generowanie polega na losowaniu (ang. sampling) kolejnych elementów z tego rozkładu. W modelach językowych elementami są tokeny, czyli fragmenty tekstu, a odpowiedź powstaje token po tokenie: każdy kolejny wybierany jest na podstawie zapytania i dotychczas wygenerowanego tekstu. Stopień losowości reguluje parametr zwany temperaturą

(ang. temperature). Dla tego samego zapytania model może więc zwrócić różne odpowiedzi, a ich powtarzalność zapewnia dopiero ustalone ziarno generatora liczb losowych (ang. seed).

Modele dyskryminatywne uczą się natomiast przypisywać dane wejściowe do kategorii, czyli modelują $p(y \mid x)$, gdzie y jest etykietą [7]. Typowym przykładem jest filtr spamu rozstrzygający, czy wiadomość jest niechcianą pocztą. Pytanie wielokrotnego wyboru wygląda jak zadanie klasyfikacji, jednak model generatywny nie wybiera klasy, lecz generuje tekst, w którym znajduje się litera odpowiedzi. Jego odpowiedź jest więc wynikiem losowania, a nie deterministycznej decyzji klasyfikatora.

Treści tworzone przez modele generatywne mogą mieć różną postać. Według [6] są to między innymi tekst, obrazy i wideo, mowa i muzyka oraz kod źródłowy. Niniejsza praca dotyczy wyłącznie tekstu.

Generatywna sztuczna inteligencja jest wykorzystywana w edukacji, nauce, medycynie, biznesie i codziennej pracy z informacją, między innymi do odpowiadania na pytania, streszczania i tłumaczenia tekstów, wyjaśniania zagadnień oraz pomocy w programowaniu. Szczególnie istotne dla tej pracy są zadania wymagające rozumowania i wiedzy specjalistycznej, takie jak rozwiązywanie zadań matematycznych i odpowiadanie na pytania medyczne, na których oparto badania opisane w dalszej części pracy. W tych dziedzinach wiarygodność odpowiedzi ma duże znaczenie, co prowadzi do ograniczeń omówionych w następnej podsekcji.

2.1.3 Ograniczenia generatywnej sztucznej inteligencji

Mimo szerokich możliwości generatywna sztuczna inteligencja ma ograniczenia, które wynikają bezpośrednio ze sposobu jej działania.

Najbardziej znanym z nich są halucynacje (ang. hallucinations), czyli generowanie treści brzmiących wiarygodnie, lecz nieprawdziwych. Wyróżnia się wśród nich halucynacje dotyczące faktów, gdy treść jest sprzeczna z weryfikowalną wiedzą o świecie, oraz halucynacje dotyczące wierności, gdy odpowiedź odbiega od polecenia lub kontekstu [8]. Ponieważ model losuje tokeny zgodnie z wyuczonym rozkładem i nie sprawdza prawdziwości treści, odpowiedź błędna wygląda tak samo jak poprawna, a przekonujący ton utrudnia wykrycie błędu. Ma to duże znaczenie w dziedzinach takich jak medycyna, w których błędna informacja może prowadzić do szkody.

Z tym wiąże się brak samoweryfikacji. Model nie posiada wbudowanego mechanizmu sprawdzania własnej odpowiedzi, a poproszenie go o ponowną ocenę własnego rozumowania nie musi prowadzić do poprawy. Pojedynczy model jest więc jednocześnie autorem i jedynym recenzentem odpowiedzi.

Kolejnym ograniczeniem jest wrażliwość na sformułowanie polecenia. Słaby prompt, czyli ogólnikowy lub pozbawiony kontekstu, zwykle daje odpowiedź niepełną lub błędną, natomiast dobrze zbudowany, precyzujący zadanie i oczekiwany format, poprawia wynik. Nawet drobne zmiany formy polecenia, niezmieniające jego sensu, mogą zmienić odpowiedź.

Ograniczenia te, w szczególności brak samoweryfikacji, stanowią punkt wyjścia dla koncepcji współpracy wielu modeli, opisanej w kolejnych rozdziałach.

2.2 Duże modele językowe

2.2.1 Definicja i architektura

Duże modele językowe to nic innego jak sieci neuronowe o bardzo dużej liczbie parametrów, trenowane na ogromnych zbiorach tekstów. Opierają się one na architekturze Transformer

[9], która umożliwia efektywne modelowanie zależności w długich sekwencjach. Dzięki temu mogą one rozpoznawać i interpretować złożone teksty. Przed ich pojawieniem się, do tego typu zadań stosowano rekurencyjne sieci neuronowe (RNN [10]), które zaczęły mieć trudności wraz z zwiększeniem modeli i długości sekwencji. Przez potrzebę wykonywania sekwencyjnych obliczeń były one mało efektywne.

Pojawienie się obecnych modeli uznaje się za wyraźny przełom w przetwarzaniu języka naturalnego (NLP) i w całej dziedzinie sztucznej inteligencji (SI)[5]. Odpowiedzialna za to jest wcześniej wspomniana architektura Transformer. Opublikowana w 2017 roku praca "Attention Is All You Need"[9] wprowadziła mechanizm samo-uwagi (ang. self-attention), który pozwala modelom dynamicznie ważyć znaczenie poszczególnych słów w kontekście całej sekwencji. Poza tym architektura ta umożliwia równoległe przetwarzanie danych, co znacząco zwiększa efektywność treningu i wnioskowania w porównaniu do wcześniejszych podejść opartych na RNN. Od tego momentu zaczęto tworzyć coraz to większe modele. Liczba parametrów współczesnych modeli wynosi miliardy, a w największych przypadkach sięga bilionów (10^{12}). Przełomem okazały się rodziny modeli takie jak GPT, LLaMA i Gemini, które pokazały, że wzrost rozmiaru, ilości danych i mocy obliczeniowej przekłada się na jakość działania [5].

2.2.2 Uczenie dużych modeli językowych

Trening dużego modelu językowego przebiega w dwóch głównych etapach: wstępnym uczeniu (ang. pre-training) i uczeniu końcowym (ang. post-training).

Podczas pre-trainingu model uczy się na dużych zbiorach tekstu bez etykiet, a jego zadanie jest jedno: przewidzieć kolejny token na podstawie poprzednich. Taki trening wystarcza, by model nauczył się budować zdania poprawne pod względem składni. Matematycznie model uczy się rozkładu prawdopodobieństwa warunkowego $P(w_t \mid w_1, \ldots, w_{t-1})$, czyli prawdopodobieństwa kolejnego tokenu przy danym kontekście. Dopiero przy generowaniu trzeba z tego rozkładu wybrać token. Najprostsza strategia, zachłanne dekodowanie (ang. greedy decoding), w każdym kroku bierze token najbardziej prawdopodobny w danym kontekście:

$$w_t = \arg\max_w P(w \mid w_1, w_2, \ldots, w_{t-1})$$

Jakość przewidywań mierzy funkcja straty, którą w trakcie uczenia minimalizujemy:

$$L = -\sum_{t=1}^{T} \log P(w_t \mid w_1, w_2, \ldots, w_{t-1})$$

W efekcie sieć przyswaja zasady gramatyki i ogólną budowę języka. Powstaje model bazowy, który przewiduje najbardziej prawdopodobną kontynuację tekstu. To nie to samo co wykonanie polecenia użytkownika, bo cel uczenia (przewidzenie kolejnego tokenu) różni się od celu, jakim jest pomocne i bezpieczne wykonywanie poleceń [11].

Tę lukę zmniejsza post-training. W schemacie opisanym dla modelu InstructGPT składa się on z trzech kroków [11]. Najpierw model dostraja się nadzorowanie (ang. supervised fine-tuning, SFT) na przykładach pożądanych odpowiedzi napisanych przez ludzi. Następnie ludzie oceniają kilka odpowiedzi modelu na to samo polecenie, a na tych porównaniach uczy się model nagrody (ang. reward model), który przewiduje, którą odpowiedź woli człowiek. W ostatnim kroku model dostraja się uczeniem ze wzmocnieniem tak, by maksymalizował tę nagrodę (ang. reinforcement learning from human feedback, RLHF; w InstructGPT użyto algorytmu PPO). Podobny trzyetapowy schemat opisują Feuerriegel i in. dla ChatGPT [1].

Po takim uczeniu model potrafi wykonywać polecenia wydawane przez użytkownika. Po odpowiednim sformułowaniu prompta potrafi przetłumaczyć tekst, odpowiedzieć w narzuconym formacie albo przyjąć wskazaną rolę. Dwie ostatnie umiejętności wykorzystano w tej pracy.

Dane treningowe pochodzą przeważnie z internetu i zostały napisane przez ludzi. Stąd bierze się przypuszczenie, że modele przejmują również część wzorców, według których ludzie podejmują decyzje.

2.2.3 Sterowanie, wnioskowanie i wyzwania wdrożeniowe

Inżynieria promptów

Zachowanie dużego modelu językowego można w znacznej mierze ukierunkować bez zmiany jego wag. Zajmuje się tym inżynieria promptów (ang. prompt engineering): dobór instrukcji, kontekstu, formatu odpowiedzi i przykładów tak, by model dał wynik zgodny z oczekiwaniami. Kosztuje to znacznie mniej niż ponowne trenowanie lub dostrajanie modelu, bo wymaga tylko zmiany tekstu wejściowego. Dlatego prompting jest zwykle pierwszym narzędziem, po które sięga się przy dopasowywaniu modelu do konkretnego zadania.

Chain-of-Thought

Chain-of-Thought (CoT) to technika, która zamiast prosić o samą odpowiedź, skłania się model do rozbicia złożonego problemu na logiczne kroki pośrednie i dopiero na ich podstawie sformułowania wyniku [3]. Dzięki rozłożeniu zadania na mniejsze kroki zwiększa się czytelność i staranność rozumowania modelu. Model przestaje ”zgadywać”końcowy wynik. Dzięki temu łatwiej radzi sobię z problemami wymagającymi kilku następujących po sobie operacji. Technika ta świetnie sprawdza się w arytmetyce, logice czy planowaniu. Jako efekt uboczny uzyskujemy też jawny zapis toku rozumowania, który można przejrzeć i ocenić.

Samo CoT ma jednak słabość: zwykle łączy się je z dekodowaniem zachłannym (ang. greedy decoding), w którym w każdym kroku wybierany jest najbardziej prawdopodobny token. Powstaje wtedy jedna ścieżka rozumowania i jeden błąd w jej środku przesądza o wyniku. Metoda Self-Consistency (CoT-SC) zastępuje ten mechanizm wielokrotnym próbkowaniem: model generuje wiele różnych ścieżek rozumowania, zamiast zatrzymać się na jednej, zachłannej[12][13]. Następnie wybierana jest odpowiedź najbardziej spójna, czyli taka, do której prowadzi najwięcej niezależnych ścieżek

Cenę za tę poprawę płaci się w obliczeniach. Każde dodatkowe próbkowanie to kolejne wywołanie modelu, więc koszt i czas rosną proporcjonalnie do liczby generowanych ścieżek. W zastosowaniach lokalnych, gdzie zasoby sprzętowe są ograniczone, ma to realne znaczenie przy wyborze liczby próbek.

Halucynacje

Mimo imponujących możliwości duże modele językowe mają ograniczenia. Najczęściej wskazuje się wśród nich halucynacje, czyli generowanie błędnych lub niemających oparcia w faktach treści [5]. Problem nie polega wyłącznie na tym, że model się myli. Błędne informacje podaje on tym samym płynnym, zdecydowanym tonem co poprawne, więc z samej formy odpowiedzi trudno ocenić, czy można jej zaufać. Wynika to ze sposobu działania modelu: wybiera on tekst najbardziej prawdopodobny w danym kontekście, a nie sprawdza go z wiarygodnym źródłem wiedzy.

Ryzyko rośnie w systemach złożonych z kilku współpracujących modeli lub agentów. Halucynacja jednego z nich może wywołać efekt kaskadowy: kolejne elementy systemu przyjmują błędny wynik jako pewnik i budują na nim dalsze rozumowanie, przez co błąd się powiela i wzmacnia [8]. Im dłuższy łańcuch przetwarzania, tym trudniej wskazać, w którym miejscu pojawił się pierwotny fałsz.

2.3 Mechanizmy współpracy

Ograniczenia opisane w poprzednich sekcjach, przede wszystkim halucynacje i brak samoweryfikacji, skłoniły badaczy do łączenia kilku wywołań modeli językowych w jeden system. Zamiast polegać na pojedynczej odpowiedzi, taki system zbiera kilka odpowiedzi, a następnie je porównuje, ocenia albo uzgadnia. Uczestnikami mogą być kolejne wywołania tego samego modelu albo różne modele. Każde dodatkowe wywołanie zwiększa koszt obliczeniowy, więc poszczególne mechanizmy różnią się nie tylko skutecznością, lecz także ceną.

2.3.1 Głosowanie

Najprostszym mechanizmem jest głosowanie większościowe. Kilka modeli albo kilka wywołań tego samego modelu rozwiązuje zadanie niezależnie, a za wynik przyjmuje się odpowiedź wskazaną najczęściej. Jest to ta sama idea co w metodzie Self-Consistency, rozszerzona na osobne wywołania i różne modele. W badaniach tej metody dokładność rosła wraz z liczbą agentów biorących udział w głosowaniu [12]. Modele nie wymieniają tu jednak informacji. Każdy pracuje w izolacji, a mechanizm jedynie zlicza odpowiedzi, więc błąd popełniony przez większość zostaje utrwalony. Przy dwóch uczestnikach głosowanie traci sens, bo w razie rozbieżności nie ma większości.

2.3.2 Weryfikacja odpowiedzi przez drugi model

Drugi mechanizm rozdziela role. Jeden model generuje rozwiązanie, a drugi je sprawdza: ocenia poprawność, wskazuje błędy i ewentualnie proponuje poprawkę. W ten sposób autor odpowiedzi przestaje być jej jedynym recenzentem. Ma to znaczenie, bo w badaniach modele proszone o poprawienie własnego rozumowania bez zewnętrznej informacji zwrotnej zwykle nie poprawiały wyniku, a niekiedy go pogarszały [14]. Słabością tego schematu jest asymetria. Weryfikator też może się mylić, a jego ocena nie podlega kontroli, więc błędna krytyka może zepsuć poprawną odpowiedź.

2.3.3 Debata

Debata usuwa tę asymetrię. W jej podstawowym schemacie kilka modeli najpierw rozwiązuje zadanie niezależnie, a następnie w kolejnych rundach otrzymuje odpowiedzi pozostałych uczestników i aktualizuje własną [15]. Każdy model jest więc jednocześnie autorem i recenzentem: broni swojego rozwiązania albo zmienia je pod wpływem argumentów pozostałych. Debata kończy się po ustalonej liczbie rund lub wcześniej, gdy odpowiedzi staną się zgodne. W badaniach schemat ten poprawił wyniki w zadaniach arytmetycznych i matematycznych oraz ograniczył halucynacje w pytaniach o fakty.

Skuteczność debaty zależy od różnorodności uczestników. Model, który nabrał pewności co do swojej odpowiedzi, przestaje generować nowe argumenty, nawet gdy odpowiedź jest

błędna. Zjawisko to nazwano degeneracją myśli (ang. Degeneration-of-Thought) [16]. Kilka instancji tego samego modelu dzieli te same luki w wiedzy, więc łatwo osiągają zgodność także w błędzie. Różne modele, trenowane na innych danych, częściej mylą się w różnych miejscach, a wtedy rozbieżność ich odpowiedzi sygnalizuje, że zadanie wymaga dalszej dyskusji. Debata nie chroni jednak przed efektem kaskadowym opisanym przy halucynacjach. Jeśli jeden model przekonująco przedstawi błędne rozumowanie, drugi może je przejąć, a zgodność odpowiedzi nie gwarantuje ich poprawności.

Rozdział 3

Technologie i narzędzia

Istotny był wybór odpowiednich technologii i narzędzi. Ważne było aby system był wydajny i jego poszczególne komponenty łatwe do wdrożenia dzięki odpowiednim udokumentowaniu. Celem tego rozdziału jest przedstawienie użytych w projekcie technologii oraz uzasadnienie ich wyboru.

3.1 Wybrane języki programowania

Do realizacji projektu zdecydowano się na język programowania Python, który dzięki swojej prostocie, szybkości implementacji oraz sporej liczbie bibliotek, był najlepszym wyborem z pośród innych języków. Dodatkowym atutem jest jego popularność pośród istniejących już rozwiązań uczenia maszynowego. Stanowiło to dodatkową inspirację podczas projektowania systemu.

3.2 Biblioteki

W projekcie wykorzystano szereg bibliotek, które skróciły czas implementacji, ograniczając potrzebę pisania kodu od podstaw i zapewniając sprawdzone rozwiązania typowych problemów programistycznych. Obok modułów biblioteki standardowej Pythona (json, random, re, subprocess) użyto bibliotek zewnętrznych, wymienionych w pliku requirements.txt:

requests

Biblioteka do wysyłania zapytań HTTP, uznawana w ekosystemie Pythona za standardowe narzędzie do tego celu. Udostępnia prosty interfejs do wysyłania żądań oraz odbierania i przetwarzania odpowiedzi, w tym danych w formacie JSON. W projekcie służy do komunikacji z interfejsem API platformy Ollama: klasa klienta wysyła zapytanie do punktu końcowego czatu, a następnie odbiera wygenerowaną odpowiedź modelu. Obsłużono przy tym wyjątki zgłaszane przez bibliotekę, dzięki czemu problem z połączeniem nie przerywa działania całego systemu.

huggingface_hub

Oficjalna biblioteka kliencka do współpracy z repozytorium Hugging Face Hub, w którym publikowane są modele i zbiory danych wykorzystywane w uczeniu maszynowym. Umożliwia pobieranie plików oraz zarządzanie lokalną pamięcią podręczną, dzięki czemu raz pobrane

dane nie muszą być ściągane ponownie. W projekcie nie jest wywoływana bezpośrednio — korzysta z niej biblioteka datasets, pobierając za jej pośrednictwem zbiory danych z repozytorium.

datasets

Biblioteka rozwijana przez Hugging Face, przeznaczona do ładowania i przetwarzania zbiorów danych. Zapewnia jednolity interfejs dostępu do danych niezależnie od ich pierwotnego formatu oraz zajmuje się ich pobraniem i lokalnym buforowaniem.

pandas

Biblioteka do analizy i przetwarzania danych tabelarycznych, której podstawową strukturą jest obiekt DataFrame. Pozwala wygodnie filtrować, grupować i agregować dane, a także sterować sposobem ich prezentacji. W projekcie wykorzystano ją na etapie analizy wyników: zapisane wyniki eksperymentów są wczytywane do obiektu DataFrame, a następnie zestawiane i podsumowywane, co umożliwia porównanie skuteczności poszczególnych konfiguracji systemu.

pyarrow

Biblioteka będąca implementacją Apache Arrow w języku Python. Apache Arrow definiuje kolumnowy format reprezentacji danych w pamięci, który umożliwia ich wydajne przetwarzanie i odczyt. W projekcie nie jest wywoływana bezpośrednio — pełni rolę silnika odczytującego pliki w formacie Parquet, w którym udostępniane są zbiory danych z repozytorium Hugging Face. Z tego silnika korzystają biblioteki datasets oraz pandas.

fastparquet

Kolejna biblioteka do odczytu i zapisu plików w formacie Parquet, czyli kolumnowym formacie przechowywania danych, który dobrze je kompresuje i umożliwia szybki dostęp do wybranych kolumn. Pełni tę samą funkcję co pyarrow i stanowi dla niej alternatywę, dzięki czemu pliki Parquet można wczytać niezależnie od tego, który z silników jest dostępny w środowisku. Podobnie jak pyarrow nie jest wywoływana w kodzie wprost, lecz wykorzystywana przez biblioteki wyższego poziomu.

sympy

Biblioteka do obliczeń symbolicznych, która pozwala przetwarzać wyrażenia matematyczne, przekształcać je i porównywać ze sobą. W projekcie służy do sprawdzania równoważności odpowiedzi matematycznych, ponieważ ta sama wartość może zostać zapisana na wiele sposobów, np. jako ułamek zwykły lub dziesiętny. Wykorzystano ją w module oceniającym w dwóch miejscach: przy porównywaniu odpowiedzi modelu z odpowiedzią wzorcową oraz przy ustalaniu, czy modele biorące udział w debacie doszły do zgodnego rozwiązania.

3.3 Wybrane modele językowe

Dobór modeli podyktowany był założeniami projektu. System opiera się na współpracy dwóch modeli. Jej wpływ na jakość odpowiedzi podlega badaniu, dlatego potrzebne były modele

pochodzące od różnych twórców, a nie dwie instancje tego samego modelu. Ponadto modele musiały z przyzwoitą dokładnością rozwiązywać zadania z obu dziedzin, na których oparto badania, czyli matematyki i medycyny. Kolejnym wymogiem było, aby były to modele o otwartych wagach (ang. open-weight), dostępne w bibliotece Ollamy i możliwe do uruchomienia lokalnie. Ostatnim kryterium było ograniczenie sprzętowe – oba modele musiały mieścić się w pamięci operacyjnej komputera, na którym prowadzono badania (szczegóły opisano w sekcji 3.5). W ramach tego ograniczenia wybierano modele o możliwie największej liczbie parametrów, ponieważ jest ona jednym z czynników wpływających na jakość generowanych odpowiedzi.

Tabela 3.1 zestawia modele rozważane na etapie wyboru. Wytłuszczono modele użyte w projekcie.

Tabela 3.1: Modele rozważane w projekcie

Model	**Twórca**	**Parametry**	**Decyzja i powód**
gemma4:12b	**Google**	**12 mld**	**Wybrany.** Najnowsza generacja rodziny Gemma, dobra jakość na zadaniach obu dziedzin, mieści się w limicie pamięci.
qwen3.5:9b	**Alibaba**	**9 mld**	**Wybrany.** Najnowsza generacja rodziny Qwen, dobra jakość na zadaniach obu dziedzin, inna rodzina niż Gemma.
llama3.1:8b	Meta	8 mld	Odrzucony. Wyraźnie słabsza jakość odpowiedzi.
qwen2.5:7b	Alibaba	7 mld	Odrzucony. Jakość przeciętna oraz starsza generacja względem Qwen3.5.
gemma2:9b	Google	9 mld	Odrzucony. Jakość przeciętna oraz starsza generacja względem Gemma 4.
gemma2:2b	Google	2 mld	Odrzucony. Częstsze błędy i trudności z zachowaniem wymaganego formatu odpowiedzi.
phi4-mini	Microsoft	3,8 mld	Odrzucony. Częstsze błędy i trudności z zachowaniem wymaganego formatu odpowiedzi.
deepseek-r1:14b	DeepSeek-AI	14 mld	Odrzucony. Bardzo dobre wyniki w matematyce, zbliżone do wybranych modeli, ale wyższe zużycie pamięci.

Wybrane zostały modele gemma4:12b (Google) oraz qwen3.5:9b (Alibaba). Pochodzą od różnych twórców i należą do różnych rodzin, co pozwala na zróżnicowane odpowiedzi i rozumowanie. Mają przy tym zbliżony rozmiar, dzięki czemu ewentualne różnice w wynikach nie wynikają wyłącznie z liczby parametrów. Oba reprezentowały najnowszą generację swoich rodzin w momencie wyboru.

Wybór opierał się na wstępnych próbach przeprowadzonych na działającej aplikacji, a nie na systematycznym porównaniu. Jakość wybranych modeli zbadano w rozdziale 5.

3.4 Narzędzia

Git

Git to narzędzie służące do zarządzania wersjami projektu, umożliwiające monitorowanie wprowadzanych zmian w kodzie oraz równoczesną pracę wielu osób nad tym samym projektem.

W projekcie pełnił funkcję podstawowego systemu kontroli wersji. Każdy członek zespołu pracował na oddzielnej gałęzi (ang. branch), a po zakończeniu prac wprowadzone zmiany były łączone z główną gałęzią projektu (ang. main branch). Takie podejście ułatwiało współpracę, ograniczało ryzyko występowania konfliktów oraz pozwalało na zachowanie uporządkowanej, liniowej historii zmian.

GitHub

GitHub to platforma wykorzystująca system Git, która pozwala na przechowywanie oraz zarządzanie repozytoriami znajdującymi się na zdalnych serwerach. Oprócz podstawowych funkcji oferuje również możliwość wzajemnego przeglądania i weryfikowania kodu przez członków zespołu, a także automatycznego uruchamiania procesów sprawdzających poprawność projektu, takich jak wykonywanie testów czy weryfikacja kompilacji kodu w ramach CI (ang. Continuous Integration). Funkcja przeglądu kodu pozwalała zespołowi na bieżąco kontrolować jakość wprowadzanych zmian, wykrywać potencjalne błędy oraz utrzymywać kod w odpowiednim stanie.

Ollama API

Ollama to oprogramowanie umożliwiające lokalne uruchamianie i zarządzanie modelami językowymi typu LLM (ang. Large Language Model). Pobiera modele z własnej biblioteki, przechowuje je w formacie GGUF i udostępnia przez lokalny serwer HTTP, domyślnie pod adresem `localhost:11434` [17].

Większość modeli w bibliotece Ollamy jest dostępna w wersjach kwantyzowanych. Proces kwantyzacji polega na zapisaniu wag modelu z mniejszą precyzją niż ta, w której model trenowano: zamiast 16 bitów na wagę (FP16 lub BF16) używa się np. 8 albo 4 bitów. Format GGUF pochodzi z projektu llama.cpp, na którym opiera się Ollama, i obsługuje kilka schematów kwantyzacji. Ich nazwy kodują liczbę bitów i metodę: Q8_0 oznacza 8 bitów na wagę, a Q4_K_M 4-bitową kwantyzację typu k-quant w wariancie średnim (ang. medium). W schematach k-quant wagi dzieli się na niewielkie bloki, a każdy blok ma własny współczynnik skali. Dzięki temu wartości odtwarza się z mniejszym błędem niż przy jednej skali dla całego tensora. Wariant M zapisuje z wyższą precyzją część tensorów, na które model jest szczególnie wrażliwy, więc średnio wypada niecałe 5 bitów na wagę.

Zysk jest przede wszystkim pamięciowy. Model o 12 mld parametrów zajmuje w FP16 ok. 24 GB, a w Q4_K_M około 7–8 GB, co mieści się w pamięci konsumenckiej karty graficznej albo w RAM zwykłego komputera. Kwantyzacja przyspiesza też generowanie. Przy każdym kolejnym tokenie trzeba odczytać z pamięci wszystkie wagi, a mniejsze wagi to mniej danych do przesłania. Ceną jest utrata jakości, mierzona zwykle wzrostem perpleksji względem modelu niekwantyzowanego. Przy 8 bitach różnica jest pomijalna, przy 4 bitach niewielka, a przy

schematach 2- i 3-bitowych wyraźna. Mniejsze modele tracą na kwantyzacji więcej niż duże. Domyślne tagi w bibliotece Ollamy wskazują zwykle wersje Q4_K_M, a inne warianty pobiera się, podając tag z nazwą kwantyzacji. W projekcie użyto wersji domyślnych.

W projekcie Ollama jest jedynym kanałem dostępu do modeli, więc przechodzi przez nią każde zapytanie systemu. Modele działały na lokalnej infrastrukturze, bez przesyłania danych do zewnętrznych usług. Dawało to pełną kontrolę nad wersją modelu i parametrami generowania oraz brak opłat za zapytania. Ceną są ograniczenia sprzętowe: rozmiar modeli (tu rzędu 9–12 mld parametrów) i czas generowania zależą od pamięci i mocy obliczeniowej komputera.

Klasa klienta komunikuje się z Ollamą za pomocą biblioteki `requests`. Wysyła żądanie POST w formacie JSON do punktu końcowego `/api/chat`, które zawiera:

- `model` – nazwę modelu, który ma odpowiedzieć;
- `messages` – listę wiadomości z rolami: `system` niesie opis roli modelu, a `user` treść zadania;
- `options` – parametry generowania: temperaturę, maksymalną liczbę generowanych tokenów (`num_predict`) oraz opcjonalnie ziarno losowości (`seed`), służące do uzyskiwania powtarzalnych wyników;
- `stream` ustawione na `false` – odpowiedź wraca jako jeden obiekt JSON po zakończeniu generowania, a nie jako strumień fragmentów, co upraszcza dalsze przetwarzanie;
- `format` – ogranicza odpowiedź do poprawnego obiektu JSON albo do podanego schematu JSON. Ułatwia to parsowanie i kontrolę struktury, ale nie gwarantuje poprawności merytorycznej odpowiedzi. Dokumentacja zaleca, by w prompcie również polecić modelowi odpowiedź w formacie JSON, i tak robią prompty w projekcie;
- `think` – włącza lub wyłącza tryb rozumowania w modelach, które go obsługują. Rozumowanie wraca wtedy w osobnym polu odpowiedzi.

Odpowiedź taka zawiera treść wiadomości, a także metadane: liczbę tokenów promptu (`prompt_eval_count`) i odpowiedzi (`eval_count`) oraz czasy ładowania modelu, analizy i generowania promptu, podawane w nanosekundach. Z nich wylicza się szybkość generowania w tokenach na sekundę jako $\texttt{eval_count}/\texttt{eval_duration} \cdot 10^9$ [17]. Klasa zwraca te dane razem z treścią i z zmierzonym czasem całego żądania, co pozwala ocenić koszt obliczeniowy każdej odpowiedzi. Żądanie ma limit czasu 600 s, ponieważ generowanie na lokalnym sprzęcie bywa długie. Błędy sieci i błędy HTTP są przechwytywane, więc awaria jednego żądania nie przerywa całego eksperymentu.

Przed uruchomieniem system sprawdza poleceniem `ollama list`, czy model jest dostępny lokalnie, a jeśli go brakuje, pobiera go poleceniem `ollama pull`. Domyślnie Ollama trzyma model w pamięci jeszcze przez pięć minut po ostatnim żądaniu (parametr `keep_alive`) [17]. Dlatego po zakończeniu pracy model jest wyładowywany poleceniem `ollama stop`, co zwalnia RAM dla kolejnego modelu.

3.5 Wymagania sprzętowe

Badania prowadzono na komputerze MacBook Pro z procesorem Apple M4 Pro i 24 GB pamięci operacyjnej. Pamięć ta jest zunifikowana (współdzielona przez procesor i układ graficzny). Model uruchomiony na GPU zajmuje zatem te same 24 GB, z których korzystają system operacyjny i pozostałe programy.

Model językowy musi w całości mieścić się w pamięci na czas generowania odpowiedzi. Główną część tej pamięci zajmują wagi, czyli parametry modelu. W niekwantyzowanej wersji bf16, czyli 16-bitowej, według biblioteki Ollamy zajmowałyby 24 GB dla modelu gemma4:12b [18] i 19 GB dla qwen3.5:9b [19]. Pierwszy z nich zająłby więc całą pamięć komputera. Problem rozwiązuje kwantyzacja, czyli zapis wag z mniejszą liczbą bitów kosztem niewielkiej straty dokładności, co zmniejsza zapotrzebowanie na pamięć. Oba użyte modele mają kwantyzację Q4_K_M (kwantyzacja czterobitowa). Tabela 3.2 zestawia parametry modeli i zajmowane przez nie zasoby.

Tabela 3.2: Zasoby zajmowane przez użyte modele

Model	**Parametry**	**Kwantyzacja**	**Dysk**	**Pamięć w działaniu**	**Procesor**
gemma4:12b	11,9 mld	Q4_K_M	7,6 GB	8,1 GB	100% GPU
qwen3.5:9b	9,7 mld	Q4_K_M	6,6 GB	5,5 GB	100% GPU

Obie wartości zajętej pamięci pochodzą z polecenia `ollama ps`.

Ollama domyślnie utrzymuje model w pamięci przez 5 minut od ostatniego użycia [20] i może mieć załadowanych kilka modeli jednocześnie, jeśli mieszczą się w dostępnej pamięci [20]. W systemie modele biorące udział w debacie są wywoływane kolejno, a polecenie `ollama stop` wykonywane jest po zakończeniu całego eksperymentu. Podczas debaty obu modeli możliwa jest zatem obecność obu w pamięci, a ich łączne zapotrzebowanie, równe 13,6 GB, mieści się w dostępnych 24 GB.

Rozdział 4

Projekt systemu

4.1 Implementacja systemu przetwarzania z wykorzystaniem pojedynczego modelu LLM

4.2 Rozszerzenie systemu do wariantu z dwoma współpracującymi modelami LLM

Rozdział 5

Badania i analiza

5.1 Wybór benchmarków oraz metryk oceny jakości odpowiedzi

5.1.1 Benchmarki

5.1.2 Metryki oceny

5.2 Przeprowadzenie analizy wyników oraz porównanie jakości odpowiedzi w obu trybach działania systemu

Rozdział 6

Podsumowanie

Spis tabel

1.1 Podział odpowiedzialności autorów za poszczególne rozdziały 8

3.1 Modele rozważane w projekcie 17
3.2 Zasoby zajmowane przez użyte modele 20

Spis rysunków

Bibliografia

[1] S. Feuerriegel, J. Hartmann, C. Janiesch, P. Zschech, "Generative AI," *Business & Information Systems Engineering*, 2023. URL: https://arxiv.org/abs/2309.07930

[2] L. Fan, L. Li, Z. Ma, S. Lee, H. Yu, L. Hemphill, "A Bibliometric Review of Large Language Models Research from 2017 to 2023," *ACM Transactions on Intelligent Systems and Technology*, t. 15, nr 5, s. 1–25, 2024. URL: https://arxiv.org/abs/2304.02020

[3] Y. Yang, Q. Peng, J. Wang, Y. Wen, W. Zhang, "LLM-based Multi-Agent Systems: Techniques and Business Perspectives," *arXiv preprint arXiv:2411.14033*, 2024. URL: https://arxiv.org/abs/2411.14033

[4] Y. Cheng, Y. Xu, C. Yu, Y. Zhao, "HAWK: A Hierarchical Workflow Framework for Multi-Agent Collaboration," *arXiv preprint arXiv:2507.04067*, 2025. URL: https://arxiv.org/abs/2507.04067

[5] K.-T. Tran, D. Dao, M.-D. Nguyen, Q.-V. Pham, B. O'Sullivan, H. D. Nguyen, "Multi-Agent Collaboration Mechanisms: A Survey of LLMs," *arXiv preprint arXiv:2501.06322*, 2024. URL: https://arxiv.org/abs/2501.06322

[6] S. Feuerriegel, J. Hartmann, C. Janiesch, P. Zschech, "Generative AI," *Business & Information Systems Engineering*, t. 66, nr 1, s. 111–126, 2024. URL: https://arxiv.org/abs/2309.07930

[7] A. Y. Ng, M. I. Jordan, "On Discriminative vs. Generative Classifiers: A Comparison of Logistic Regression and Naive Bayes," w: *Advances in Neural Information Processing Systems 14 (NIPS 2001)*, 2001. URL: https://proceedings.neurips.cc/paper/2001/hash/7b7a53e239400a13bd6be6c91c4f6c4e-Abstract.html

[8] L. Huang, W. Yu, W. Ma, W. Zhong, Z. Feng, H. Wang, Q. Chen, W. Peng, X. Feng, B. Qin, T. Liu, "A Survey on Hallucination in Large Language Models: Principles, Taxonomy, Challenges, and Open Questions," *ACM Transactions on Information Systems*, t. 43, nr 2, s. 1–55, 2025. URL: https://arxiv.org/abs/2311.05232

[9] A. Vaswani, N. Shazeer, N. Parmar, J. Uszkoreit, L. Jones, A. N. Gomez, Ł. Kaiser, I. Polosukhin, "Attention Is All You Need," *Advances in Neural Information Processing Systems*, t. 30, 2017. URL: https://arxiv.org/abs/1706.03762

[10] A. Sherstinsky, "Fundamentals of Recurrent Neural Network (RNN) and Long Short-Term Memory (LSTM) Network," *Physica D: Nonlinear Phenomena*, t. 404, 2020. URL: https://arxiv.org/abs/1808.03314

[11] L. Ouyang, J. Wu, X. Jiang, D. Almeida, C. Wainwright, P. Mishkin, C. Zhang, S. Agarwal, K. Slama, A. Ray, J. Schulman, J. Hilton, F. Kelton, L. Miller, M. Simens, A. Askell, P. Welinder, P. Christiano, J. Leike, R. Lowe, "Training Language Models to Follow Instructions with Human Feedback," 2022. URL: https://arxiv.org/abs/2203.02155

[12] J. Li, Q. Zhang, Y. Yu, Q. Fu, D. Ye, "More Agents Is All You Need," *Transactions on Machine Learning Research*, 2024. URL: https://arxiv.org/pdf/2402.05120

[13] X. Wang, J. Wei, D. Schuurmans, Q. Le, E. H. Chi, D. Zhou, S. Narang, A. Chowdhery, "Self-Consistency Improves Chain of Thought Reasoning in Language Models," w: *The Eleventh International Conference on Learning Representations (ICLR 2023)*, 2023. URL: https://arxiv.org/pdf/2203.11171

[14] J. Huang, X. Chen, S. Mishra, H. S. Zheng, A. W. Yu, X. Song, D. Zhou, "Large Language Models Cannot Self-Correct Reasoning Yet," w: *The Twelfth International Conference on Learning Representations*, 2023. URL: https://arxiv.org/abs/2310.01798

[15] Y. Du, S. Li, A. Torralba, J. B. Tenenbaum, I. Mordatch, "Improving Factuality and Reasoning in Language Models through Multiagent Debate," w: *Proceedings of the 41st International Conference on Machine Learning*, 2023. URL: https://arxiv.org/abs/2305.14325

[16] T. Liang, Z. He, W. Jiao, X. Wang, i in., "Encouraging Divergent Thinking in Large Language Models through Multi-Agent Debate," w: *Proceedings of the 2024 Conference on Empirical Methods in Natural Language Processing*, 2023. URL: https://arxiv.org/abs/2305.19118

[17] Ollama. "API Reference," dostęp 2 listopada 2026. URL: https://docs.ollama.com/api

[18] Ollama. "gemma4: tagi modelu w bibliotece Ollamy," dostęp 9 października 2026. URL: https://ollama.com/library/gemma4/tags

[19] Ollama. "qwen3.5: tagi modelu w bibliotece Ollamy," dostęp 9 października 2026. URL: https://ollama.com/library/qwen3.5/tags

[20] Ollama. "Ollama FAQ," dostęp 9 października 2026. URL: https://docs.ollama.com/faq